GOLDLENS / ARCHITECTURE / 2026

GoldLens 2.1
体验优先架构

用最先进的 Web、Local-first 与 AI 技术，把“快、稳、聪明”做成默认能力

Experience · Local-first · AI-native · Durable

Version 1.1 · Proposed · 2026-10-23

优化原则

先进技术不是堆栈，
是让用户感觉不到技术。

PPR · React Compiler · ElectricSQL · Inngest · AI Gateway · Base UI

Prepared for founder / engineering / design review

DOCUMENT MAP

架构优化 · 目录

00 执行摘要 2

01 v1.0 复盘 3

02 体验优先的架构原则 4

03 v1.1 目标架构 5

04 技术雷达 6

05 体验技术栈 7

06 Local-first 数据 8

07 AI-native 层 9

08 Durable workflows 与实时 11

09 性能与可访问性预算 12

10 安全、隐私与 AI 治理 13

11 迁移影响与路线 13

12 决策与下一步 15

阅读说明

本方案面向创始人、工程与设计团队，描述从当前 MVP 到生产级 GoldLens 的目标架构、数据模型、体验规范与交付路线。所有时间与成本为显式假设，不是承诺；上线前需完成数据授权、隐私与本地合规审查。

00

执行摘要

最先进的技术，不是让架构更复杂，而是让用户感觉不到技术。

从“能算”到“好用到离不开”：快、稳、聪明，且随时能退。

v1.0 的技术方案是正确的：Next.js 模块化单体、Postgres、严格的数据边界、18 周交付。但它仍然偏“稳健的 2024 架构”：前端只做到 RSC，数据只做到 API + 缓存，异步只做到队列 + worker，AI 只是可选项。对于“体验最好的现代程序”，这还不够。

v1.1 的目标不是把技术名词堆满，而是把三类体验做成默认能力：**瞬时**（计算与页面切换无等待）、**离线**（弱网/地铁/门店里也能用）、**聪明**（自然语言、扫描、自动填表，但不替用户做投资决定）。为此，我们升级五件事：Next.js 16 的 PPR/React Compiler、Base UI + View Transitions、ElectricSQL local-first、Inngest durable workflows、AI Gateway + 结构化工具调用。

表 0-1 · v1.1 的五个关键升级

层	v1.0	v1.1	用户感知
渲染	RSC + 客户端缓存	PPR + React Compiler + View Transitions	首屏更快、切市场像原生
数据	API + Redis + localStorage	ElectricSQL 读同步 + IndexedDB outbox	离线可看、写入即时
异步	QStash/BullMQ + 独立 worker	Inngest durable workflows	报价/提醒不丢、可回放
智能	无 AI	AI Gateway + 工具调用 + 结构化输出	一句话算账、拍收据填表
界面	shadcn/ui + Radix	Tailwind v4 + Base UI + Motion	更细腻、更可访问

本轮结论

采用“Experience-first 模块化单体”：Next.js 16（PPR/RSC/Server Actions）+ ElectricSQL 读同步 + Inngest 工作流 + AI Gateway。所有“先进”技术都必须通过 2 周 spike 验证，并保留回退到 v1.0 方案的能力。

0.1 三个不做

- 不做微前端、不拆微服务、不上 Kubernetes：在 10 万 MAU 之前，这些只会增加故障面。

- 不做“AI 预测金价/买卖信号”：AI 只做界面与数据录入，不做投资判断。
- 不做 3D/WebGL 炫技首屏：黄金的“高级感”来自克制、排版与数据可信，不来自帧率。

01

v1.0 复盘

正确，但还可以更“贴手”。

v1.0 的架构没有错，它解决的是“从 0 到 1 的可靠性”。v1.1 要解决的是“从 1 到 10 的体验密度”：用户不再容忍等待、刷新、重复输入；他们希望像用原生 App 一样用 Web。

表 1-1 · v1.0 与 v1.1 的差异

维度	v1.0	v1.1	为什么
首屏	RSC + 流式	PPR：静态 shell 先到，动态价格后流式注入	LCP 与感知速度
交互	客户端状态 + 请求	Server Actions + 乐观更新 + View Transitions	减少等待与跳变
数据	API 拉取 + 缓存	ElectricSQL shape 同步 + IndexedDB	离线与即时
异步	队列 + 自建 worker	Inngest step functions	可观测、可重放
AI	无	AI Gateway + 工具调用	录入与问答
组件	Radix/shadcn	Base UI + Tailwind v4	可访问性与样式性能
移动	PWA	PWA + Expo 原生壳（P3）	推送/分享/触感
交付	18 周 / 3.5 FTE	20 周 / 4 FTE（含 2 周 spike）	先进技术需要验证

1.1 必须保留的资产

- 计算引擎与测试向量：这是产品的“真理”，不能被 UI 或 AI 替代。
- 数据可追溯原则：来源、时间、置信度、规则版本，继续保留。
- 优雅降级：快照、DEMO、过期标记；先进技术不能消灭降级。
- “无托管/无交易/无建议”的边界：越先进，越要克制。
- 现有 CI、冒烟测试、静态 Demo：作为回退基线。

1.2 为什么现在升级

三个条件同时成熟：Next.js 16 的 PPR/Cache Components 让“静态 + 动态”可以在同一页共存；ElectricSQL/IndexedDB 让 Local-first 不再是研究项目；AI SDK 6 + Gateway 让结构化抽取和工具调用可以工程化。再加上 2026 年的用户已经被原生 App 和 AI 产品训练过，等待 2 秒就会流失。

02

体验优先的架构原则

先进技术必须服务于“少等待、少输入、少出错”。

2.1 六条原则

- **瞬时优先**：确定性计算（单位、纯度、到手价）在本地完成，不等网络；网络只负责“最新价格”和“写操作”。
- **本地优先读、服务端权威写**：价格/报价/公开数据用 shape 同步；持仓/提醒/订阅以服务端版本为准，客户端只做乐观 UI。
- **AI 是界面，不是神谕**：AI 负责“把话/图片变成结构化输入”，计算仍由 core 工具完成；不输出建议。
- **先进但可拔插**：Inngest、ElectricSQL、AI Gateway 都包在 adapter 后；任一处不达标，可回退，不重写业务。
- **性能是功能**：LCP、INP、CLS、JS 预算进入 CI；不通过不上线。
- **默认可访问**：键盘、读屏、对比度、减少动效、44px 触控区，和暗色主题一起做。

2.2 体验的四个时刻

表 2-1 · 关键时刻

时刻	用户感受	技术手段
打开	“怎么这么快”	PPR、CDN、字体子集、AVIF、Speculation Rules
输入	“不用等”	本地计算、乐观结果、防抖、View Transitions
比较	“看得懂”	成本瀑布、价差、回购、排名理由
回来	“它记得我”	local-first、提醒、持仓、个性化单位

03

v1.1 目标架构

少一个 worker，多一层同步，多一个 AI 网关。

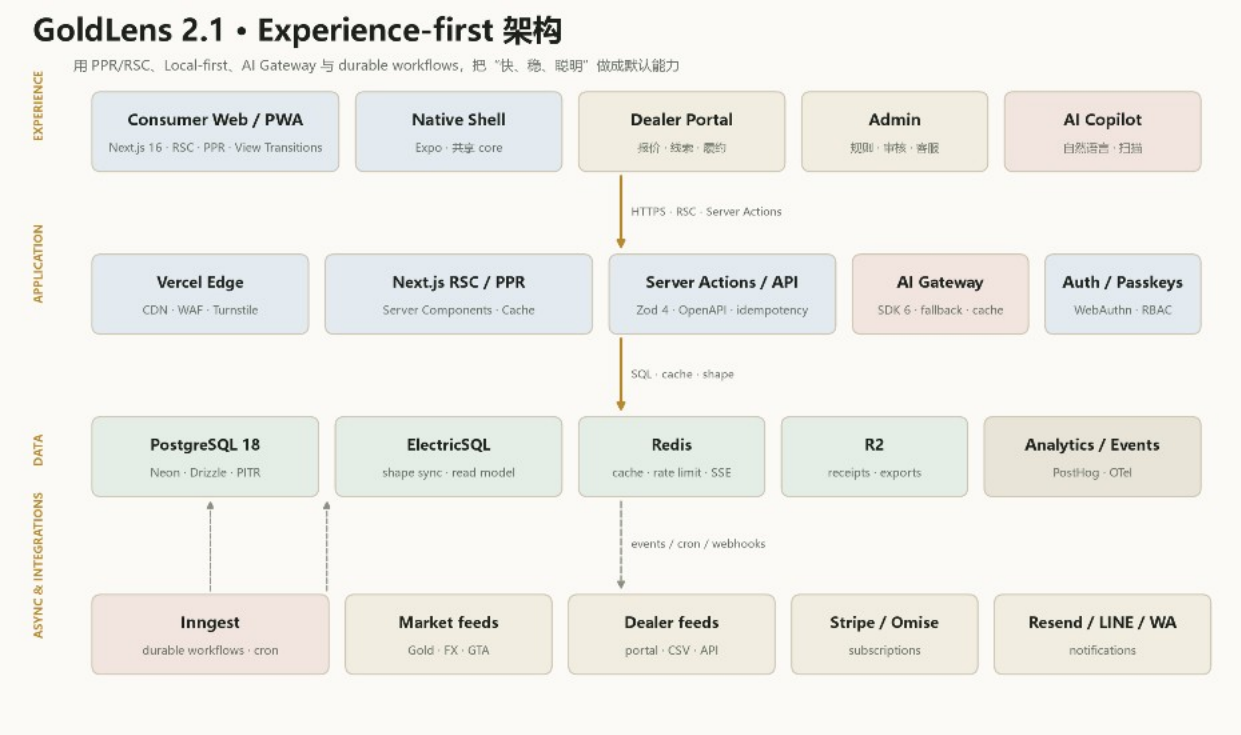


图 3-1 · GoldLens 2.1 Experience-first 架构：四层、一个 Inngest、一个 AI Gateway。

3.1 分层职责

表 3-1 · v1.1 分层

层	职责	关键变化
Experience	Web/PWA、Expo 壳、Dealer、Admin、AI Copilot	PPR/View Transitions/Base UI；Copilot 作为入口而非独立产品
Application	Edge、RSC、Server Actions、AI Gateway、Auth	去掉自建 worker；AI 走 Gateway；写操作走 Actions
Data	Postgres、ElectricSQL、Redis、R2、Analytics	增加 shape 同步；私有数据不进公共 shape
Async & Integrations	Inngest、行情、Dealer、支付、通知、观测	Inngest 取代队列 + worker；事件驱动

架构简化

v1.0 的“Next.js + 独立 worker + QStash/BullMQ”在 v1.1 变成“Next.js + Inngest”。少一个常驻服务、少一套队列运维，durable steps 自带重试、回放与可观测性。

3.2 新增与移除

表 3-2 · 架构变化

变化	内容	原因
新增	ElectricSQL shape 同步	公开价格/报价即时、离线
新增	IndexedDB outbox	弱网写入不丢
新增	Inngest durable workflows	报价/提醒/计费可靠
新增	AI Gateway + SDK 6	模型路由、缓存、降级、观测
新增	Expo native shell（P3）	推送/分享/触感
移除	自建 worker 进程	Inngest 托管执行
移除	QStash/BullMQ 队列	Inngest 事件与 step
保留	Postgres 作为唯一事实源	审计与事务

04

技术雷达

先进不是“全都要”，而是知道什么该试、什么该等。

表 4-1 · GoldLens 技术雷达（2026 Q4）

状态	技术	判断
Adopt	Next.js 16 App Router / PPR / Cache Components	首屏与动态价格共存，直接采用
Adopt	React 19 + React Compiler	减少手写 memo，提升交互一致性
Adopt	TypeScript strict + Zod 4 + Drizzle	端到端类型与契约
Adopt	Tailwind v4 + Base UI + shadcn/ui	设计系统与可访问性基线
Adopt	Motion + View Transitions API	页面转场与数字变化
Adopt	Postgres 18 + Neon + Redis + R2	事实源与缓存/对象存储
Adopt	Inngest	durable workflows，替代自建 worker
Trial	ElectricSQL	读同步；2 周 spike 验证私有 shape/离线
Trial	AI SDK 6 + AI Gateway	结构化抽取/工具调用；先做 2 个场景
Trial	Expo native shell	PWA 验证留存后再做
Trial	Speculation Rules / scroll-driven animations	渐进增强，失败不影响功能
Hold	微前端 / Module Federation	团队规模不匹配
Hold	Kubernetes / 服务网格	Vercel + Inngest 足够
Hold	GraphQL	RSC + REST/OpenAPI 更简单
Hold	Convex / Supabase 全家桶	SQL/审计/可迁移优先

状态	技术	判断
Hold	3D/WebGL 首屏	性能与可访问性代价高
Hold	自研 CRDT 同步引擎	先用 ElectricSQL + 服务端权威写

05

体验技术栈

让 Web 像原生，让原生像同一个产品。

5.1 Web 与交互

表 5-1 · 体验栈

能力	选择	用法
渲染	Next.js 16 PPR + RSC	静态 shell 先到，价格/个性化流式注入
编译	React Compiler	自动 memo；减少手写 useMemo/useCallback
样式	Tailwind v4 + CSS variables	OKLCH 色彩、容器查询、subgrid
组件	Base UI + shadcn/ui	Dialog/Select/Tabs/Popover，无障碍基线
动效	Motion + View Transitions	市场切换、结果卡、数字滚动
预取	Speculation Rules + Link prefetch	hover/viewport 预取，减少等待
字体	可变字体 + 子集	中文按需子集，避免 2MB 字体
图片	AVIF/WebP + responsive sizes	门店/收据/分享卡
PWA	Serwist + IndexedDB	离线 shell、最近价格、持仓
原生	Expo SDK 54 + Expo Router	推送/分享/触感；共享 core 包

5.2 现代 Web 平台能力

- View Transitions：市场切换、计算器结果更新不闪屏；失败时回退普通渲染。
- Speculation Rules：对“计算/市场/门店”链接预渲染，首屏点击接近 0ms。
- Container Queries + Subgrid：结果卡在侧栏/移动端自动重排，不写两套布局。
- OKLCH + color-mix：暗/亮主题与金色状态色一致，不出现“脏金”。
- @starting-style + anchor positioning：弹层/提示从触发点自然出现。
- Scroll-driven animations：价格区间的进度条随滚动解释，不影响主线程。

5.3 页面体验规格

表 5-2 · 页面与“感知”

页面	关键体验	技术
Calculate	输入即结果，无“计算中”	本地 core + 乐观结果 + 数字动效
Compare	切市场不刷新	View Transitions + shape 同步
Quotes	报价到达实时插入	SSE/Inngest + 乐观列表
Portfolio	离线也能看	IndexedDB + 本地估值
Dealer	报价表单自动填充	AI 抽取 + 结构化校验
Learn	秒开、可分享	PPR + ISR + OG 图

5.4 为什么不是 Astro / SvelteKit / 原生优先

Astro 对内容站更轻，但 GoldLens 的核心是登录态、计算器、报价与实时数据，Next.js 的 RSC/PPR/Actions 更完整；SvelteKit 性能优秀，但团队与生态（shadcn/Base UI/Expo 共享）更适合 React；原生优先会拖慢验证，PWA 先跑 8 周，再用 Expo 补推送与触感。

06

Local-first 数据

读要瞬时，写要可靠，私有数据不乱跑。

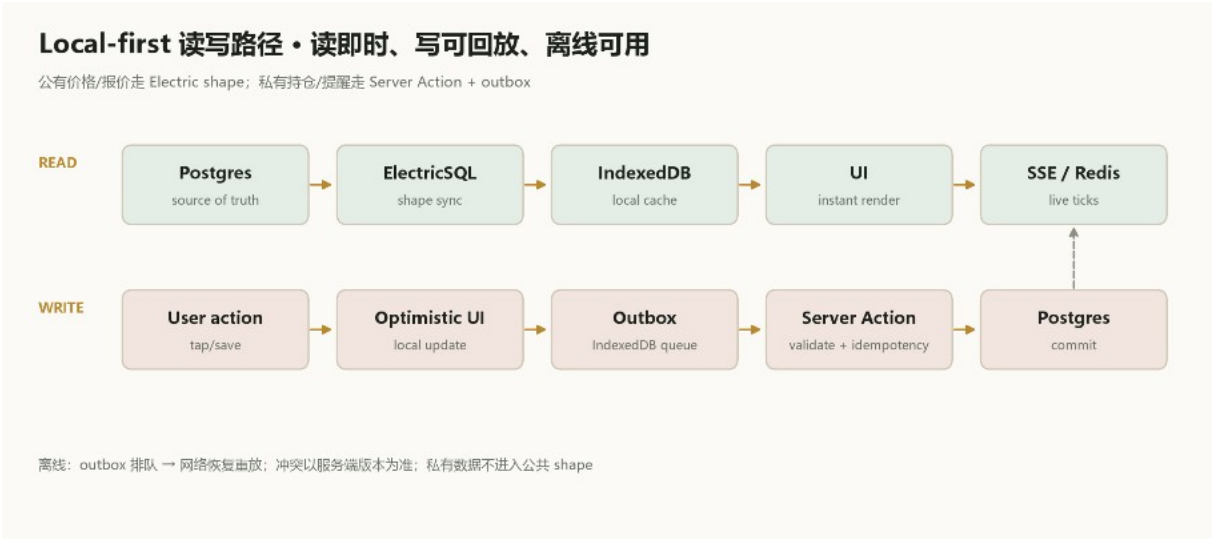


图 6-1 · Local-first 读写路径：公开数据 shape 同步，私有写入 outbox + Server Action。

6.1 同步范围

表 6-1 · 数据同步策略

数据	读	写	原因
最新价格/汇率	ElectricSQL shape + SSE	仅服务端	公开、可缓存、强时效
Dealer 报价	ElectricSQL shape	Dealer Portal / API	公开可比较、带过期
持仓	Server Action + IndexedDB	Server Action + outbox	私有、需审计
提醒	Server Action + IndexedDB	Server Action + outbox	私有、需服务端调度
订阅	Server Action	Stripe webhook	不可离线修改
收据	R2 签名 URL	上传 + 服务端扫描	隐私与病毒扫描

6.2 写入与冲突

- 客户端生成 client_id + idempotency_key；Server Action 以唯一键去重。
- 持仓/提醒以服务端 version 为准；客户端乐观更新，失败回滚并保留草稿。
- 离线写入进 IndexedDB outbox；恢复网络后按顺序重放，冲突时提示“保留云端/保留本地”。
- 金额/重量等关键字段不做自动合并；冲突副本显式呈现。
- 公共 shape 不包含用户私有字段；私有数据走带会话的 API，避免 RLS 配置错误导致泄露。

6.3 为什么不是 Zero / Convex

Zero 的本地写体验更强，但引入新的同步服务与学习成本；Convex 的实时 DX 很好，但 SQL/审计/迁移与金融数据治理不如 Postgres 直接。v1.1 选择“ElectricSQL 读同步 + 服务端权威写”，在体验与可控性之间取平衡；Zero 作为 V2 的候选，等团队与数据模型稳定后再评估。

07

AI-native 层

让 AI 做“输入”，不让 AI 做“判断”。

AI 在 GoldLens 的定位是“降低输入成本、提升解释效率”，不是“给买卖建议”。所有涉及金额的计算都必须调用 @goldlens/core 的工具，AI 只负责把自然语言/图片/邮件转成结构化参数，并让用户确认。

表 7-1 · AI 用例

用例	输入/输出	时延目标	成本目标	护栏
自然语言算账	“1 baht 96.5 今天曼谷” → 参数	< 800ms	< $0.002/次	工具调用 core；不自由计算
收据/报价扫描	图片 → 金额/重量/纯度/日期	< 3s	< $0.01/张	字段置信度；用户确认
Dealer 邮件解析	邮件/CSV → 结构化报价	< 5s	< $0.02/条	人工复核后才上架
客服 Copilot	问题 → 带引用的回答	< 2s 首 token	< $0.01/轮	RAG + 引用；不答建议
每日摘要	价格变化 → 个性化摘要	批处理	< $0.001/人	模板 + 数据；不预测

7.1 AI Gateway 与工程化

- AI SDK 6 + AI Gateway：统一 provider、超时/重试/降级、缓存、用量与成本观测。
- 结构化输出用 Zod schema；解析失败自动重试一次，再回退人工。
- 工具 allowlist：calculate_quote、convert_unit、lookup_dealer、get_price；禁止任意代码/网络。
- Prompt 版本化，评估集 ≥ 200 条（收据/邮件/问法），每次模型/提示变更跑回归。
- PII 脱敏：手机号、邮箱、证件号在进入模型前替换；使用零保留/区域化端点。
- 成本上限：按用户/功能/日预算；超限降级到规则解析或人工。
- 人工复核：任何进入报价/持仓的 AI 字段，首次必须用户确认。

7.2 AI 治理

表 7-2 · AI 风险与控制

风险	控制
幻觉/算错	工具调用 core；输出 schema；二次校验；不自由生成数字
Prompt injection	把收据/邮件当不可信数据；工具 allowlist；系统提示隔离；输出过滤
隐私	PII 脱敏、零保留、区域端点、最小化上传
成本失控	预算、缓存、批处理、降级、按功能开关
监管	不输出投资建议；显著 AI 标识；人工复核；审计日志
供应商锁定	AI Gateway 抽象；prompt/评估与 provider 解耦

08

Durable workflows 与实时

少一个 worker，多一层可靠。

Inngest 把“报价分发、过期、提醒、计费、AI 抽取”变成可观测的 step functions：每一步有输入/输出、重试、超时、回放；不需要维护队列与 worker。Next.js 只负责请求/响应，长任务交给 Inngest。

表 8-1 · Inngest 函数

函数	触发	关键步骤	幂等/重试
ingest.prices	cron 60s	fetch → validate → persist → cache → SSE	source + observed_at
quotes.dispatch	event	load request → select dealers → notify → wait	request_id
quotes.expire	cron 5m	mark expired → rerank → notify user	quote_id
alerts.check	cron 60s	load alerts → price → compare → notify	alert_id + window
billing.dunning	Stripe event	retry → grace → downgrade	invoice_id
ai.extract	event	download → redact → model → schema → review	file_hash + prompt_version
digest.daily	cron 8am	aggregate → summarize → send	user_id + date

8.1 实时价格

表 8-2 · 实时策略

场景	方案	降级
价格看板	SSE `/events/prices`，30s/变化推送	5s 轮询
报价到达	Inngest → SSE/通知	刷新列表
Dealer 后台	SSE + 乐观列表	手动刷新
离线	IndexedDB 最近快照	显示“离线/时间戳”

09

性能与可访问性预算

“快”要可测量，“好看”要可用。

表 9-1 · 体验预算

指标	目标	验证
LCP（中端安卓 4G）	< 1.8s	Lighthouse CI + RUM p75
INP	< 100ms	RUM + Playwright
CLS	< 0.05	Lighthouse
TTFB（缓存）	< 300ms	Checkly
首屏 JS	< 120KB gzip	bundle analyzer
路由切换	< 150ms 感知	View Transitions + 预取
可访问性	WCAG 2.2 AA	axe + 键盘 + 读屏
对比度	≥ 4.5:1	token 审查

9.1 设备分层

- 低端安卓（Moto G4 / 4x CPU 降速）：计算器可用、图表降级为静态 SVG、AI 功能按需加载。
- 中端 iPhone/Android：完整交互、View Transitions、PWA 安装。
- 桌面：并排对比、键盘快捷键、Command Palette。
- 弱网/离线：最近价格、计算器、持仓可读；写操作排队。

9.2 可访问性与国际化

- WCAG 2.2 AA：焦点可见、键盘可达、错误可读、触控 ≥ 44px。
- prefers-reduced-motion：关闭转场/数字滚动，保留状态变化。
- next-intl + ICU：zh/en/th/ar；RTL 用 logical properties，不写 left/right。
- 数字/货币/单位按 locale 格式化；价格永远带时间戳与来源。
- SEO：PPR/ISR、hreflang、JSON-LD、OG 图；程序化落地页。

10

安全、隐私与 AI 治理

越“聪明”，越要限制它。

表 10-1 · v1.1 新增风险

风险	场景	控制
Local-first 泄露	设备丢失/共享设备	不缓存收据原图；敏感数据加密；会话过期清理
Shape 越权	公共 shape 混入私有字段	shape 白名单；私有数据走 API；自动测试
AI 注入	收据/邮件含恶意指令	不可信数据隔离；工具 allowlist；输出 schema
AI 误填	金额/重量识别错	置信度阈值；用户确认；人工复核
供应商	AI/同步/工作流锁定	adapter + 出口数据 + 回退方案
成本	AI/同步/工作流费用失控	预算、限额、缓存、按功能开关

10.1 安全基线（在 v1.0 之上）

- Passkeys/WebAuthn 优先；邮箱 magic link 兜底；敏感操作二次验证。
- Turnstile + 速率限制 + 设备指纹，保护计算/询价/AI 接口。
- CSP 收紧到 self + 必要域名；AI 输出不进 innerHTML。
- AI 供应商零保留/区域端点；PII 脱敏；prompt/response 审计保留 30 天。
- 依赖供应链：Dependabot/CodeQL/secret scanning；AI SDK 固定 minor。

11

迁移影响与路线

20 周，不是 18 周；多出的 2 周买“确定性”。

表 11-1 · 迁移影响

v1.0 组件	v1.1 处理	风险
QStash/BullMQ + worker	迁移到 Inngest 函数	重写调度/重试逻辑
TanStack Query 缓存	保留 + ElectricSQL shape	双写期一致性
localStorage 持仓	IndexedDB + Server Action	迁移/冲突
shadcn/Radix	Base UI + 保留样式 token	组件行为差异
无 AI	AI Gateway + 2 个用例	准确率/成本

v1.0 组件	v1.1 处理	风险
18 周计划	20 周 + 2 周 spike	排期

11.1 20 周路线

表 11-2 · 阶段与验收

阶段	周期	交付	验收
Spike	W1–2	PPR、Electric、Inngest、AI 四个验证	五项指标达标，否则回退
P0 基础	W1–4	Monorepo、core、DB、Auth、设计系统	核心向量 100%；登录/同步
P1 消费端	W3–9	计算器、市场、持仓、提醒、PWA、i18n	LCP < 1.8s；激活 ≥ 45%
P2 Dealer	W7–14	Portal、询价、Inngest、订阅、Admin	10 家；24h 响应 ≥ 70%
P3 规模化	W12–18	UAE/India、Expo 壳、性能/安全	支付 ≥ 1.5%；Lighthouse ≥ 90
Hardening	W19–20	渗透测试、压测、灾备、发布	0 critical/high；回滚演练

11.2 Spike 验收（Go/No-Go）

表 11-3 · 两周必须证明的五件事

验证	通过标准	不通过的回退
PPR/React Compiler	中端安卓 LCP < 1.8s；INP < 100ms	退回 RSC + 客户端缓存
ElectricSQL	公开 shape 同步 < 300ms；离线可读	TanStack Query + IndexedDB
Inngest	报价流程可见、可重试、可回放	保留 QStash/BullMQ worker
AI 抽取	50 张收据字段准确率 ≥ 95%	只做规则解析 + 人工
成本	AI < $0.01/次；基础设施 < $1.5k/月	限制功能/换模型

11.3 团队与预算

4 FTE × 20 周：Tech Lead/Full-stack 1、Frontend 1、Backend/Data 1、Product Designer 0.5、QA/DevOps 0.5。基础设施 500–1,500 美元/月（10 万 MAU 量级），AI 预算 200–500 美元/月（按 1 万次抽取/月估算），不含数据授权与 Dealer 地推。

11.4 风险

表 11-4 · 主要风险

风险	缓解
先进技术学习曲线	2 周 spike；每项技术一个 owner；ADR
ElectricSQL 成熟度	只用于公开读模型；私有数据走 API；可回退

风险	缓解
AI 准确率/成本	结构化输出 + 人工确认 + 预算 + 评估集
Vercel/Inngest 锁定	adapter；导出/回放；本地 dev 容器
体验过度设计	性能预算 + A/B + 可用性测试
排期延长	P0/P1 先上线，P2/P3 可裁剪

12

决策与下一步

先把“先进”验证成“可靠”，再全面铺开。

表 12-1 · ADR

ADR	决策	状态
ADR-008	Inngest 替代自建 worker + 队列	Proposed
ADR-009	ElectricSQL 读同步；写入服务端权威	Proposed
ADR-010	AI Gateway + 工具调用；AI 不做判断	Proposed
ADR-011	Base UI + Tailwind v4 + Motion	Proposed
ADR-012	20 周 / 4 FTE，含 2 周 spike	Proposed
ADR-013	不做微前端/K8s/3D/自研 CRDT	Accepted

12.1 需要确认的三件事

1. 是否接受“2 周 spike + Go/No-Go”作为先进技术的准入条件?
2. 是否接受 20 周 / 4 FTE，而不是压缩到 18 周?
3. 第一个 AI 功能选“自然语言算账”还是“收据扫描”？前者更快，后者更能体现差异。

下一步

批准后先做 2 周 spike：PPR、ElectricSQL、Inngest、AI 抽取各做一个可运行切片；第 2 周末用表 11-3 做 Go/No-Go。通过则进入 P0，不通过则按回退方案继续，不把风险带进主路径。

先进技术不是目的。让用户在曼谷的店里、在弱网的地铁里、在只会说一句英文的情况下，仍然能算清楚、拿到报价、放心离开——这才是 GoldLens 2.1 的“体验最好”。